Authx Reference - API Extra

MiddlewareOauth2

Warning

You need to install dependencies to use The Oauth2 Middleware.

```
$ pip install authx_extra
```

class authx_extra.oauth2.MiddlewareOauth2

```
MiddlewareOauth2(app, providers, public_paths=None, get_keys=None, key_refresh_minutes=None)
```

Source code in `authx_extra/oauth2.py`

```
def __init__(
    self,
    app: ASGIApp,
    providers: typing.Dict[str, typing.Dict[str, typing.Any]],
    public_paths: typing.Optional[typing.Set[str]] = None,
    get_keys: typing.Optional[typing.Callable[[typing.Any], typing.Any]] = None,
    key_refresh_minutes: typing.Optional[
        typing.Union[int, typing.Dict[str, int]]
    ] = None,
) -> None:
    self._app = app
    for provider in providers:
        _validate_provider(provider, providers[provider])
    self._providers = providers
    self._get_keys = get_keys or _get_keys
    self._public_paths = public_paths or set()

    if key_refresh_minutes is None:
        self._timeout = dict.fromkeys(providers)
    elif isinstance(key_refresh_minutes, dict):
        self._timeout = {
            provider: datetime.timedelta(minutes=key_refresh_minutes[provider])
            for provider in providers
        }
    else:
        self._timeout = {
            provider: datetime.timedelta(minutes=key_refresh_minutes)
            for provider in providers
        }

    # cached attribute and respective timeout
    self._last_retrieval: typing.Dict[str, datetime.datetime] = {}
    self._keys: typing.Dict[str, typing.Any] = {}
```

meth claims

```
claims(token)
```

Source code in `authx_extra/oauth2.py`

```python
def claims(self, token: str) -> typing.Tuple[str, typing.Dict[str, str]]:
    errors: typing.Dict[str, str] = {}
    for provider in self._providers:
        try:
            return provider, self._provider_claims(provider, token)
        except jose.exceptions.ExpiredSignatureError as e:
            # if the token has expired, it is at least from this provider.
            logger.debug("Token has expired.")
            errors = str(e)
            break
        except jose.exceptions.JWTClaimsError as e:
            logger.debug("Invalid claims")
            errors[provider] = str(e)
        except jose.exceptions.JOSEError as e:  # the catch-all of Jose
            logger.warning(e, exc_info=True)
            errors[provider] = str(e)
    raise InvalidToken(errors)
```

March 20, 2026 March 19, 2026 GitHub